Bài thực hành 05

Data Validate & Annotation

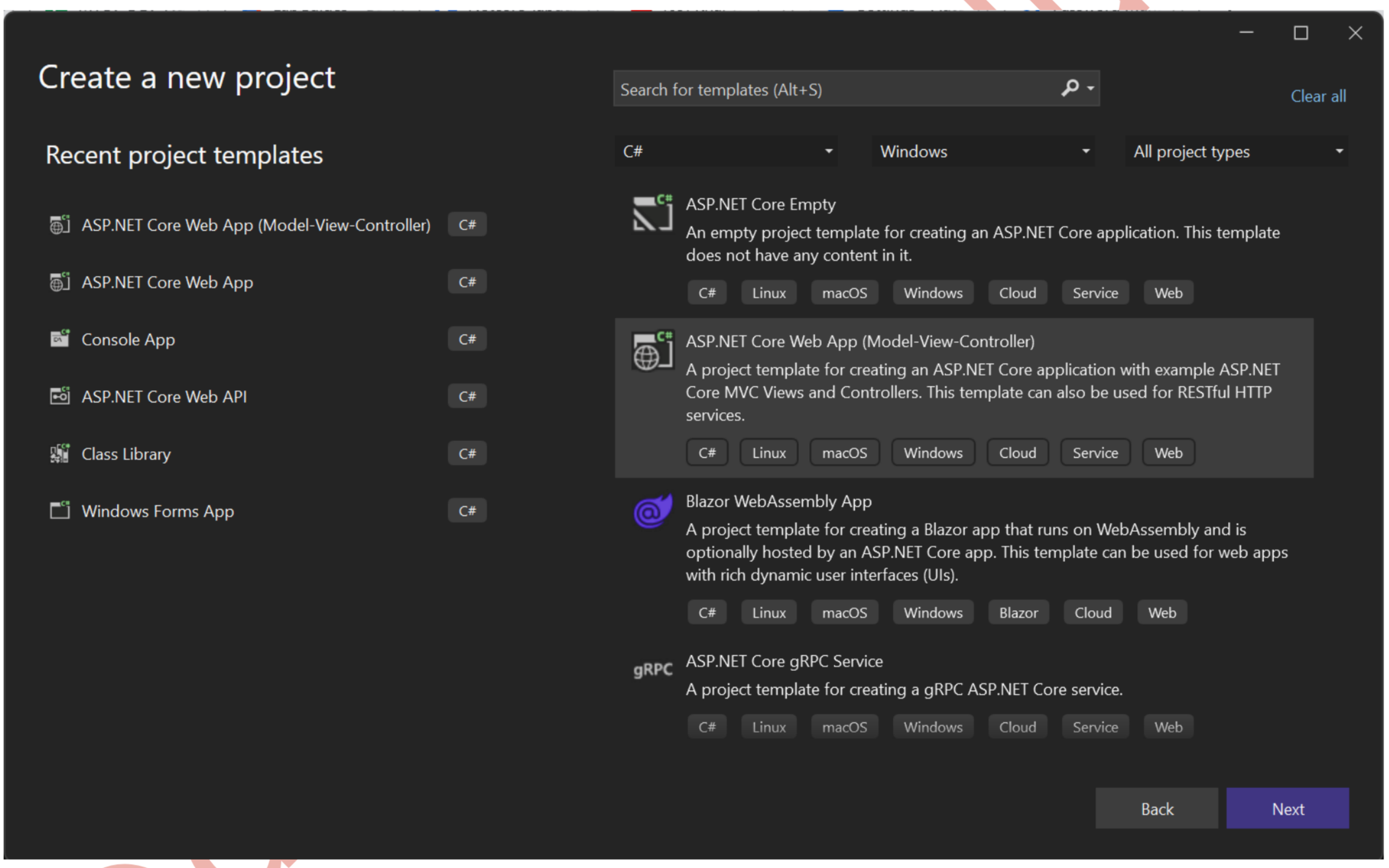
1. *Mục tiêu*

- Tạo model và validate dữ liệu sử dụng các Annotation có sẵn
- Custom Validate theo ý muốn

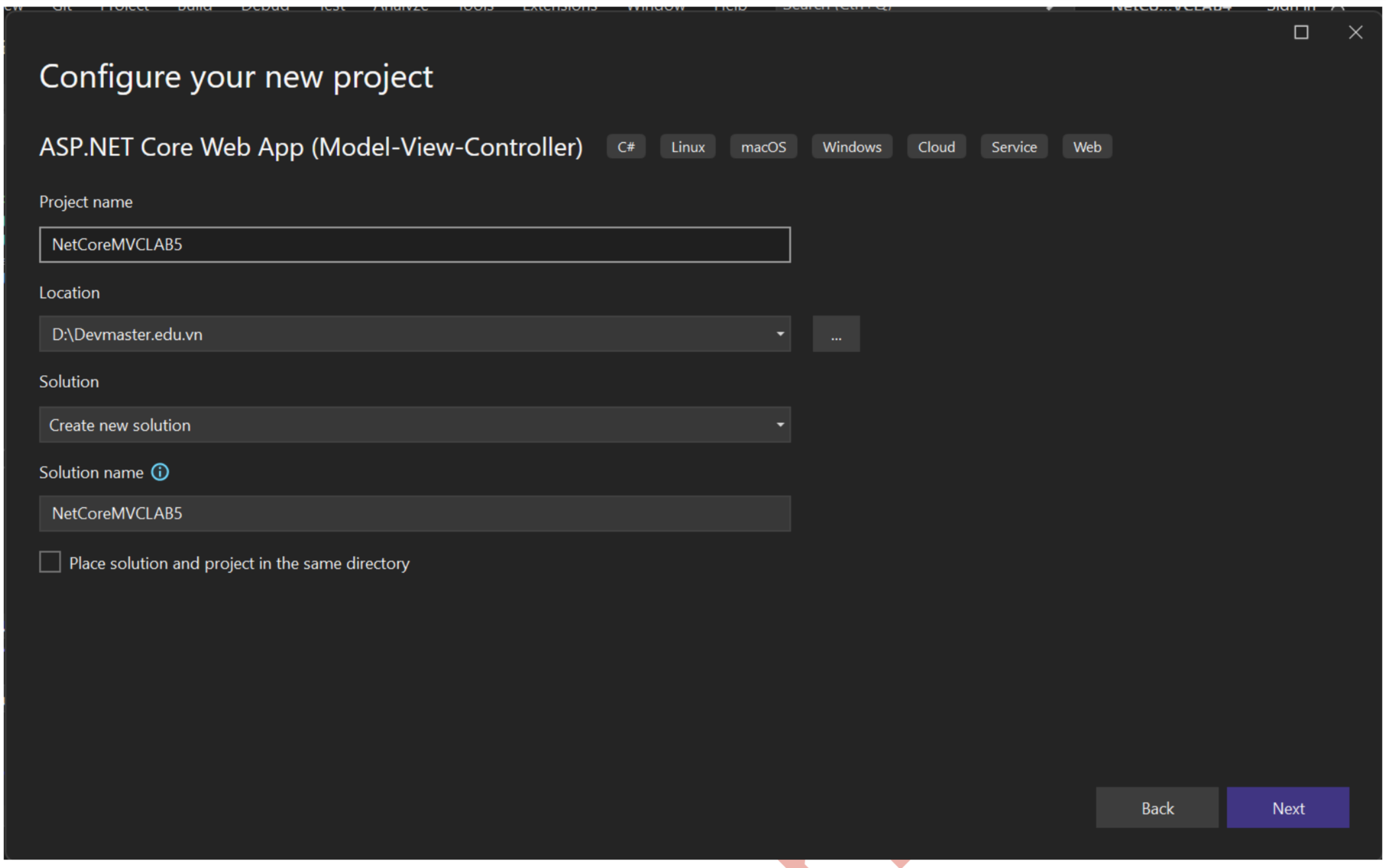
2. *Bài thực hành Step by Step (Kế thừa các lab trước)*

Bài 1: Áp dụng Annotation validate form

Bước 1: Tạo project mới và chọn Template MVC



Đặt tên là NetCoreMVCLAB5



Bước 2: Tạo Model Account với các thuộc tính sau

```
using System;
namespace NetCoreMVCLAB5.Models
{
    public class Account
    {
        [Key]
        public int Id { get; set; }
        public string FullName { get; set; }
        public string Email { get; set; }
        public string Phone { get; set; }
        public string Address { get; set; }
        public string Avatar { get; set; }
        public DateTime Birthday { get; set; }
        public string Gender { get; set; }
        public string Password { get; set; }
        public string Facebook { get; set; }
    }
}
```

Bước 3: Sử dụng các **Annotation** ràng buộc dữ liệu cho thuộc tính của model Account

Mở lại model và khai báo các rang buộc cho từng thuộc tính theo một trong các cách sau

```
using System.ComponentModel.DataAnnotations;
namespace NetCoreMVCLAB5.Models
{
    public class Account
    {
        [Key]
        public int Id { get; set; }

        [
            Display(Name = "Họ và tên"),
            Required(ErrorMessage = "Họ không được để trống"),
            MinLength(6, ErrorMessage ="Họ tên ít nhất là 6 ký tự"),
            MaxLength(20, ErrorMessage = "Họ tên tối đa 20 ký tự")
        ]
        public string FullName { get; set; }

        [Display(Name = "Địa chỉ email")]
        [Required(ErrorMessage = "Địa chỉ email không được để trống")]
        [EmailAddress(ErrorMessage = "Địa chỉ email không đúng định dạng")]
        public string Email { get; set; }
        //Các trường khác.........
    }
}
```

Khóa chính

Gộp nhiều ràng buộc trong một cặp [...]

Mỗi rang buộc là một cặp ngoặc [....]

Toàn bộ code modl Account như sau

```
using System;
using System.ComponentModel.DataAnnotations;
namespace NetCoreMVCLAB5.Models
{
    public class Account
    {
        [Key]
        public int Id { get; set; }

        [
            Display(Name = "Họ và tên"),
            Required(ErrorMessage = "Họ không được để trống"),
            MinLength(6, ErrorMessage ="Họ tên ít nhất là 6 ký tự"),
            MaxLength(20, ErrorMessage = "Họ tên tối đa 20 ký tự")
        ]
        public string FullName { get; set; }

        [Display(Name = "Địa chỉ email")]
        [Required(ErrorMessage = "Địa chỉ email không được để trống")]
        [EmailAddress(ErrorMessage = "Địa chỉ email không đúng định dạng")]
        [DataType(DataType.EmailAddress)]
        public string Email { get; set; }

        [Display(Name = "Số điện thoại")]
        [DataType(DataType.PhoneNumber)]
        [RegularExpression(@"^\(?([0-9]{3})\)?[-. ]?([0-9]{3})[-. ]?([0-
9]{4})$", ErrorMessage = "Số điện thoại không đúng định dạng")]
        [Required(ErrorMessage = "Số điện thoại không được để trống")]
        public string Phone { get; set; }

        [Display(Name = "Địa chỉ thường trú")]
        [Required(ErrorMessage = "Địa chỉ không được để trống")]
        [StringLength(35, ErrorMessage = "Địa chỉ không vượt quá 35 ký tự")]
        public string Address { get; set; }

        [Display(Name = "Ảnh đại diện")]
        public string Avatar { get; set; }

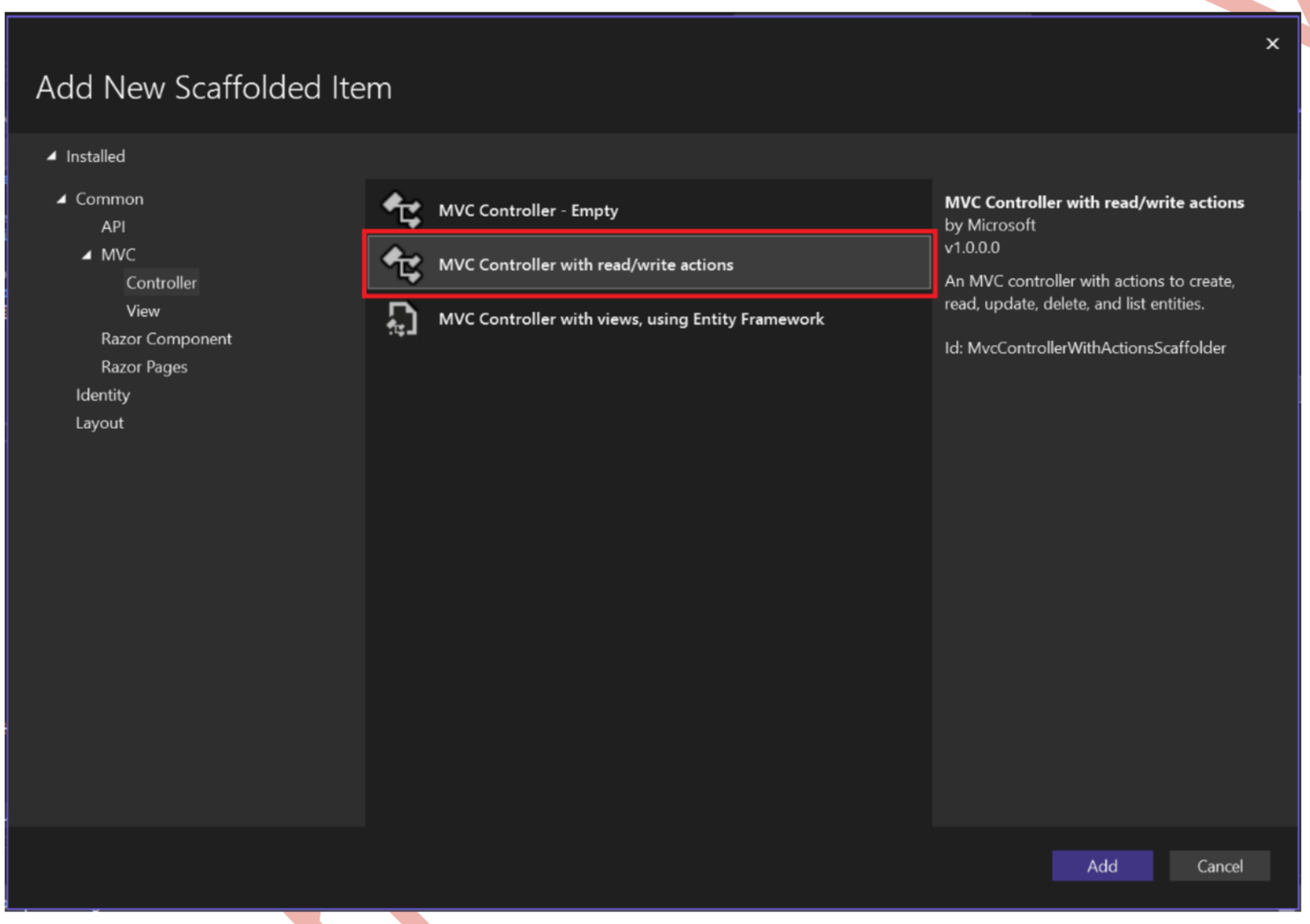
        [Display(Name = "Ngày sinh")]
        [Required(ErrorMessage = "Ngày sinh không được để trống")]
        [DataType(DataType.Date)]
        public DateTime Birthday { get; set; }
```

```
        [Display(Name = "Giới tính")]
        public string Gender { get; set; }

        [Display(Name = "Mật khẩu")]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Display(Name = "Link Facebook cá nhân")]
        [Required(ErrorMessage = "Link Facebook không được để trống")]
        [Url(ErrorMessage = "Url phải đúng định dạng bao gồm http hoặc https, tên miền VD: https://facebook.com/itvnsoft")]
        public string Facebook { get; set; }
    }
}
```

Bước 4: Tạo AccountController sử dụng template MVC Controller with read/write action



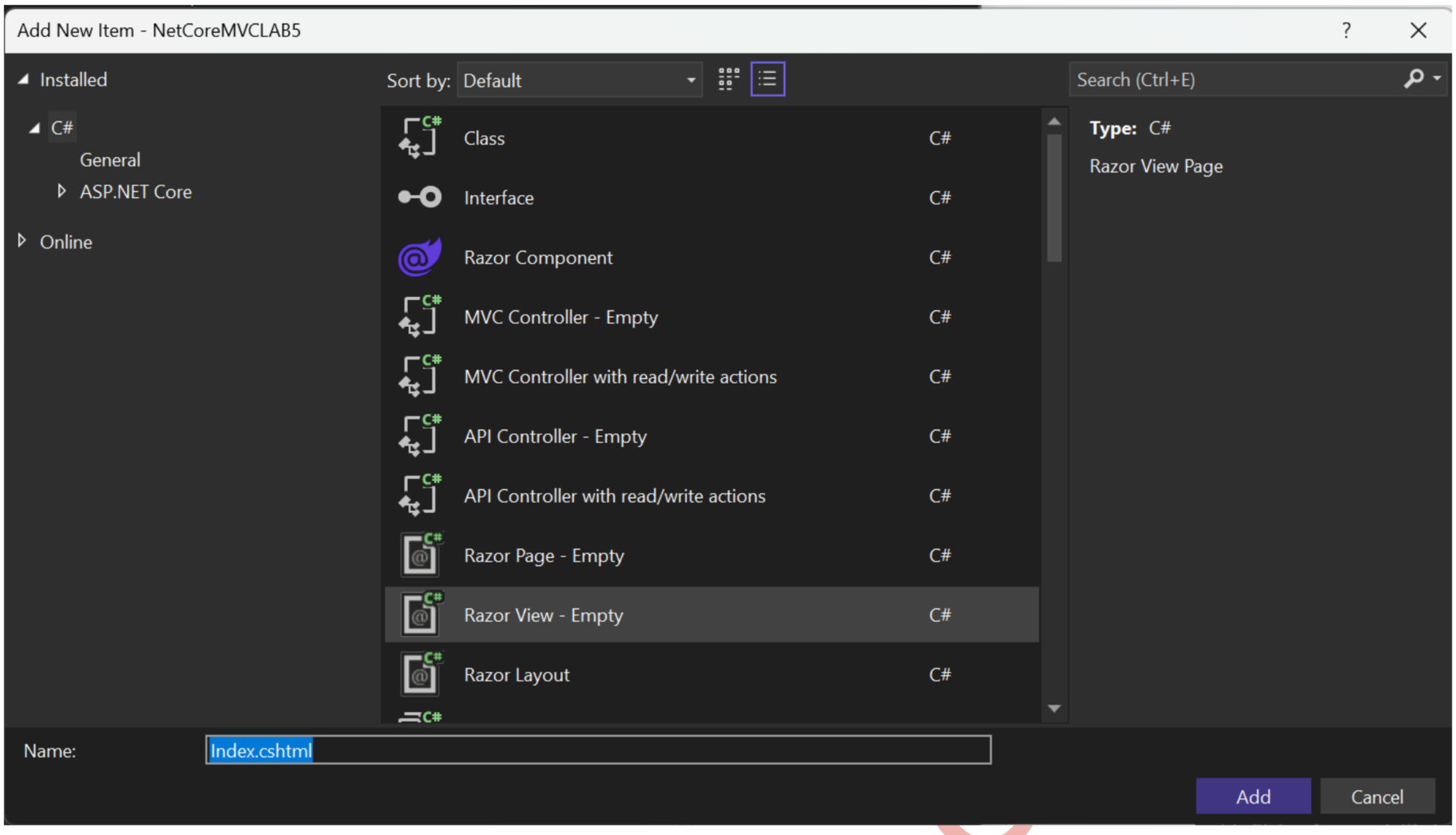
AccountController có nội dung sau khi tạo, có đủ các action cần thiết cho CRUD

Mở Action Index lên soạn code như sau

```
// GET: Account
public ActionResult Index()
{
    List<Account> accounts = new List<Account>();
    return View(accounts);
}
```

Hiển thị Danh sách

Bước 4: **AddView** cho Action Index trên đây sử dụng template List như hình dưới

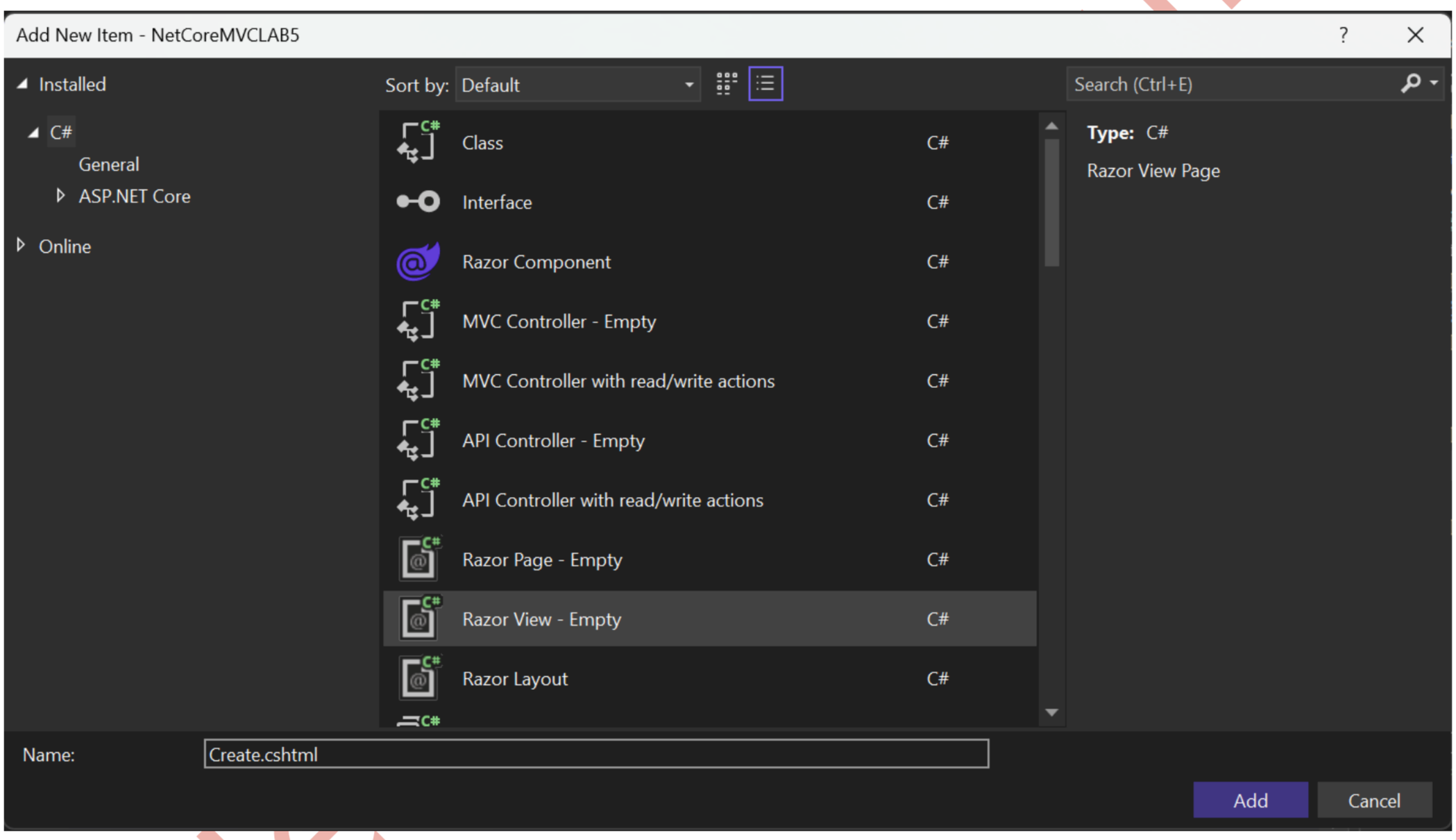


Build lại dự án chạy dự án và truy cập link có dạng

Bước 5: Tiếp tục tìm đến Action Create soạn code như sau

```
// GET: Account
public ActionResult Index()...
// GET: Account/Details/5
public ActionResult Details(int id)...
// GET: Account/Create
public ActionResult Create()
{
    Account model = new Account();
    return View(model);
}
```

Bước 6: Add View cho action Create sử dụng template Create như sau



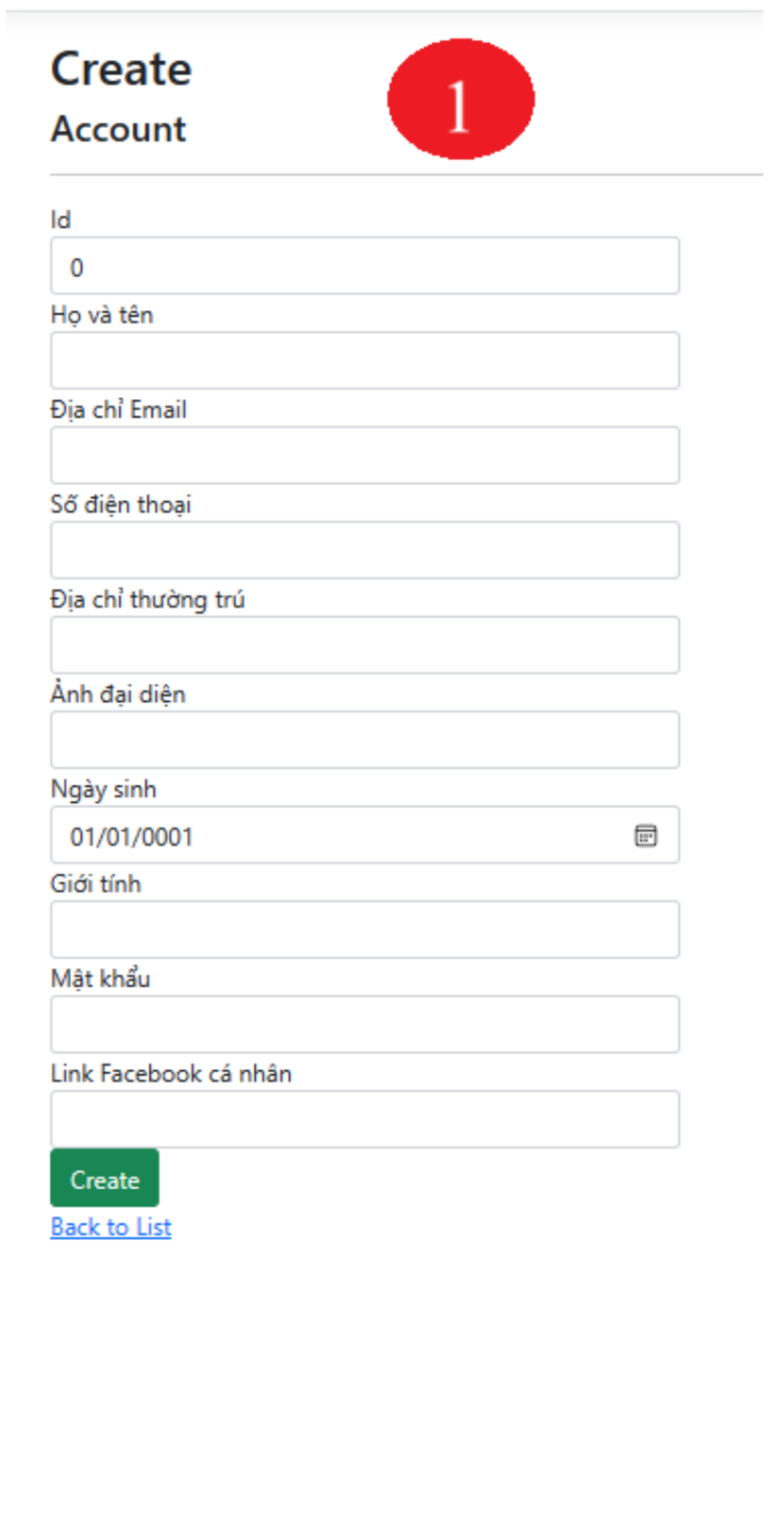
Sau khi click Add, nội dung file Create.cshtml có nội dung là form thêm mới Account

```
@model NetCoreMVCLAB5.Models.Account
@{
    ViewData["Title"] = "Create";
}
<h2>Create</h2>
<h4>Account</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="Create">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Id" class="control-label"></label>
                <input asp-for="Id" class="form-control" />
                <span asp-validation-for="Id" class="text-danger"></span>
            </div>
            <div class="form-group">
```

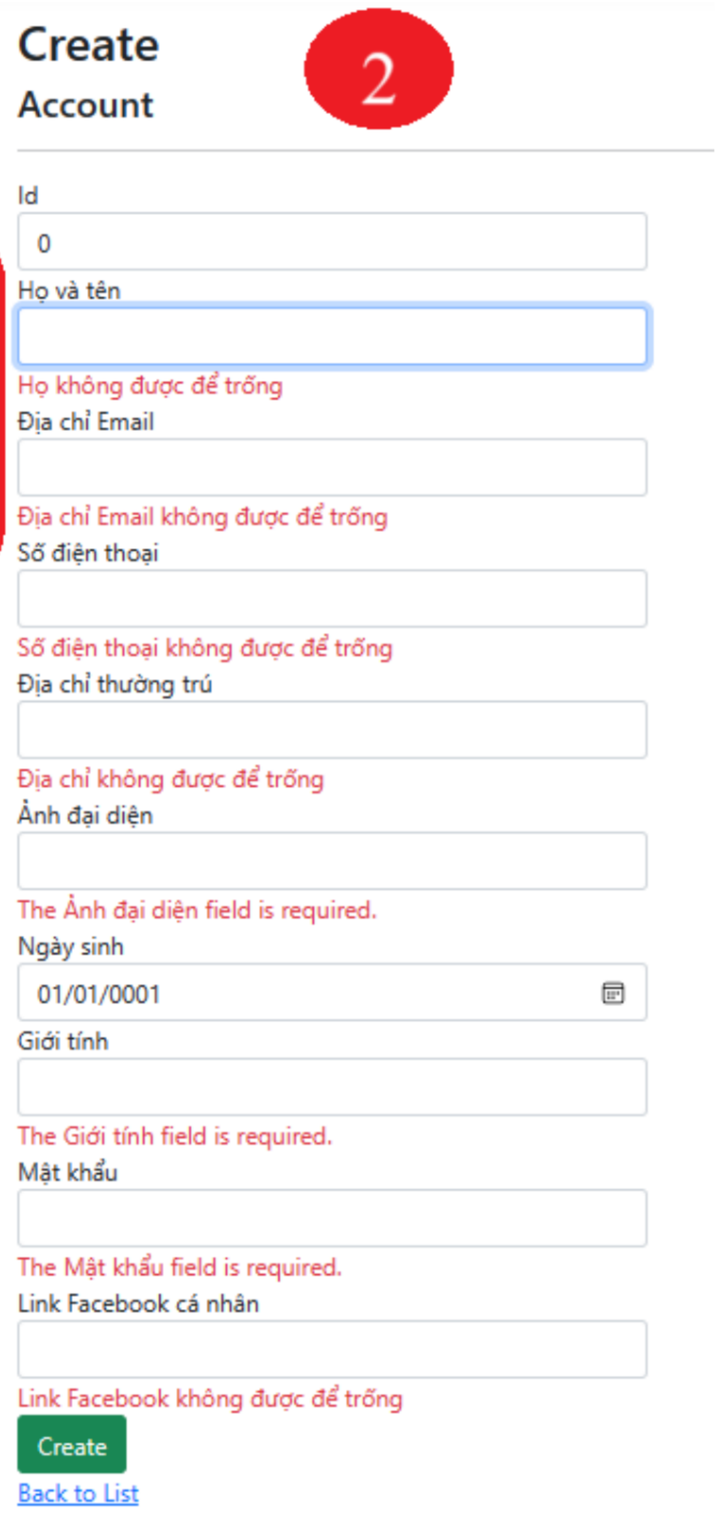
```
            <label asp-for="FullName" class="control-label"></label>
            <input asp-for="FullName" class="form-control" />
            <span asp-validation-for="FullName" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Email" class="control-label"></label>
            <input asp-for="Email" class="form-control" />
            <span asp-validation-for="Email" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Phone" class="control-label"></label>
            <input asp-for="Phone" class="form-control" />
            <span asp-validation-for="Phone" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Address" class="control-label"></label>
            <input asp-for="Address" class="form-control" />
            <span asp-validation-for="Address" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Avatar" class="control-label"></label>
            <input asp-for="Avatar" class="form-control" />
            <span asp-validation-for="Avatar" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Birthday" class="control-label"></label>
            <input asp-for="Birthday" class="form-control" />
            <span asp-validation-for="Birthday" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Gender" class="control-label"></label>
            <input asp-for="Gender" class="form-control" />
            <span asp-validation-for="Gender" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Password" class="control-label"></label>
            <input asp-for="Password" class="form-control" />
            <span asp-validation-for="Password" class="text-danger"></span>
        </div>
        <div class="form-group">
            <label asp-for="Facebook" class="control-label"></label>
            <input asp-for="Facebook" class="form-control" />
            <span asp-validation-for="Facebook" class="text-danger"></span>
        </div>
        <div class="form-group">
            <input type="submit" value="Create" class="btn btn-default" />
        </div>
    </form>
</div>
</div>
<div>
    <a asp-action="Index">Back to List</a>
</div>
@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}
```

Bước 7: Build lại dự án và truy cập link có dạng sau

https://localhost:44380/Account/Create







Bước 8: Remote validation, áp dụng cho validate thuộc tính Phone

Mở lại Account model tìm đến thuộc tính Phone

```
[Display(Name = "Số điện thoại")]
[DataType(DataType.PhoneNumber)]
[RegularExpression(@"^\(?([0-9]{3})\)?[-. ]?([0-9]{3})[-. ]?([0-
9]{4})$", ErrorMessage = "Số điện thoại không đúng định dạng")]
[Required(ErrorMessage = "Số điện thoại không được để trống")]
```

Thay thế validate **RegularExpression** thành remote như sau

```
[Display(Name = "Số điện thoại")]
[DataType(DataType.PhoneNumber)]
[Remote(action: "VerifyPhone", controller:"Account")]
[Required(ErrorMessage = "Số điện thoại không được để trống")]
public string Phone { get; set; }
```

Theo như trên thì cần khai báo một Action tên là **VerifyPhone** trong **AccountController**

Bước 9: Mở lại AccountController và khai báo action VerifyPhone

```
[AcceptVerbs("GET", "POST")]
public IActionResult VerifyPhone(string phone)
{
    Regex _isPhone = new Regex(@"^\(?([0-9]{3})\)?[-. ]?([0-9]{3})[-. ]?([0-9]{4})$");
    if (!_isPhone.IsMatch(phone))
    {
        return Json($"Số điện thoại {phone} Không đúng định dạng, VD: 0986421127 hoặc 098.421.1127");
    }

    return Json(true);
}
```

Bước 10: Build lại dự án, truy cập form thêm mới Account theo link

https://localhost:44380/Account/Create

Thử nhập sai định dạng phone





BÀI TẬP TỰ LÀM THÊM

Sinh viên Tạo dự án Net Core MVC mới hoặc kế thừa các bài trức

Khai báo 2 model là Category (Danh mục) và Product (Sản phẩm)

Model Category gồm

int Id, string Name

Model Product gồm

int Id, string Name, string Image, float Price, float SalePrice, string Description

Tất cả các thuộc tính bắt buộc nhập, ngoài ra validate thêm

Thuộc tính Name ít nhất là 6 ký tự, nhiều nhất là 150 ký tự

Thuộc tính Price phải nhỏ nhất 100000, kiểu dữ liệu text

Thuộc tính SalePrice không âm và phải nhỏ hơn giá chuẩn 10%

Thuộc tính Image phải được chọn và upload vào thư mục wwwroot/products

Thuộc tính CategoryId phải có trong một danh sách là các đối tượng của Category gồm các thuộc tính: int Id, string Name

Thuộc tính Description không được vượt quá 1500 ký tự, không chứa các từ nhạy cảm (tùy chọn, VD: die, admin, fack…)

TIếp theo sinh viên thực hiện tạo (CRUD): danh sách, form thêm mới, form chỉnh sửa, xem chi tiết, xóa ... cho Model Product

Trên form có mục (CategoryId) được chọn từ ComboBox (Select option) là các đối tượng của Category trên

Lưu ý: nội dung các tin nhắn thông báo lỗi phải được thông báo bằng tiếng việt rõ ràng và dễ hiểu

@devmaster.edu.vn